

Feature: Identificação Patrimonial e Unidades

[docs/features](#)

Ciclartech · Plataforma de Gestão de Ativos Assistivos

Identificação Patrimonial (QR Code) e Gestão de Unidades

Especificação funcional recebida do cliente em 2026-07-29. Serve de referência durante todo o desenvolvimento do módulo — mantenha este documento e o código sincronizados; se a implementação divergir da spec por decisão técnica, registre a divergência na seção "Status da implementação" abaixo, não apague o texto original.

Objetivo

Este módulo tem como objetivo:

- identificar individualmente cada ativo da plataforma;
- agilizar operações através do QR Code;
- permitir impressão em lote das etiquetas patrimoniais;
- organizar os ativos por unidade operacional;
- definir corretamente a hierarquia de usuários (Admin → Gestor → Funcionário).

1. Identificação Patrimonial

Todo ativo deve possuir: Código Patrimonial, QR Code exclusivo, Categoria resumida, Nome/logo da instituição (opcional).

QR Code nasce automaticamente ao salvar o ativo (nunca gerado manualmente).

Código Patrimonial:

- Geração automática (ex: CAD-000001, MUL-000045, AND-000012, conforme categoria)
- Código personalizado (cliente com patrimônio próprio, ex: PMSJC-2548) — sistema deve validar unicidade

Centro de Etiquetas (módulo dentro de Ativos):

- imprimir etiqueta individual
- imprimir em lote (com filtros: categoria, status, unidade, selecionar todos)
- gerar PDF
- reimprimir etiquetas
- visualizar ativos sem etiqueta impressa
- visualizar histórico de impressão
- fila de impressão ao cadastrar novos ativos
- Na ficha do ativo: "Última impressão", "Quantidade de impressões", "Reimprimir"
- Múltiplos layouts de etiqueta (Pequeno/Médio/Grande)

Localização do Ativo:

- "Localizar Ativo" com opções: Escanear QR Code / Pesquisar patrimônio / Pesquisar categoria / Pesquisar nome
- QR Code é acelerador, nunca obrigatório
- Sistema nunca mostra ações incompatíveis com status atual do ativo

Histórico por QR Code: ficha completa, timeline, movimentações, fotos, manutenções, empréstimos, devoluções.

2. Gestão de Unidades

Objetivo: permitir que um cliente possua diversas unidades físicas sob um único contrato.

Hierarquia: Owner Ciclartech → Cliente → Administrador → Gestores → Funcionários

- **Owner:** visualiza todos os clientes/contratos/métricas/planos (plataforma).
- **Administrador** (um ou mais por cliente): cadastra/edita/desativa unidades, cadastra gestores/funcionários, define permissões, visualiza TODOS os ativos/empréstimos/unidades da organização.
- **Gestor:** obrigatoriamente vinculado a uma unidade; visualiza apenas dados da própria unidade; pode cadastrar ativos, emprestar, devolver, enviar manutenção, cadastrar beneficiários, emitir relatórios da unidade, ver dashboard da unidade. NÃO visualiza dados de outras unidades.
- **Funcionário:** pertence a uma unidade; pode localizar ativos, emprestar, devolver, registrar fotos, consultar ativos. NÃO altera configurações da organização.

Cadastro de Unidade (menu Administração → Unidades): Nome, Tipo, Responsável, Telefone, E-mail, Endereço, Cidade, Estado, Observações, Status (Ativa/Inativa).

Todo ativo deve obrigatoriamente possuir uma unidade responsável.

Dashboard por Unidade: mostra contagens (total, emprestados, manutenção, disponíveis) por unidade para o Admin.

Transferência entre unidades: preparar arquitetura para futura implementação (fluxo: unidade origem → transferir → unidade destino → registrar movimentação → atualizar timeline; histórico permanece registrado).

Decisão de arquitetura: permissão de acesso, não vínculo fixo

Em vez de vincular o Gestor diretamente à Unidade, o modelo usa uma relação intermediária:

Usuário → Perfil (Admin | Gestor | Funcionário) → Permissões → Unidades permitidas

Isso permite cenários como: um gestor responsável por duas unidades, um funcionário cobrindo férias em outra unidade, um administrador com acesso a todas as unidades. A unidade deixa de ser uma limitação fixa do usuário e passa a ser uma permissão de

acesso — mais flexível, sem aumentar a complexidade de uso para quem opera o sistema no dia a dia.

Status da implementação

Feito

- **Usuario.unidades** (M2M para `core.Unidade`): substitui o vínculo fixo "Gestor pertence a uma unidade" por uma lista de unidades permitidas. Admin sempre vê tudo, independente do que estiver marcado aqui — ver `core/unidades.py`.
- **core/unidades.py**: lógica central de escopo/permissão — `unidades_visiveis(usuario)`, `unidades_do_usuario(usuario)`, `usuario_pode_operar_unidade(usuario, unidade)`. Registrado como exceção auditada em `core/tests/test_architecture.py` (precisa de `all_tenants()` porque roda fora de request context — ver comentário no próprio módulo).
- **CRUD de Unidade** (`core/views_unidades.py`, `core/urls_unidades.py`, `templates/core/unidades/*.html`): menu Administração → Unidades, restrito a Admin (`nivel_hierarquico >= NIVEL_ADMIN`). Campos conforme spec (nome, tipo, responsável, telefone, e-mail, endereço, cidade, UF, observações, ativo/inativa). Validação de nome único por tenant.
- **Atribuição de unidades ao criar usuário** (`contas/forms.py::CriarUsuarioForm`, `contas/views.py::usuarios_criar`): checkbox list de unidades ativas na tela de criação de Gestor/Funcionário.
- **Código Patrimonial automático** (`ativos/patrimonio.py`, `ativos/models.py::CategoriaAtivo.prefixo`, `ativos/forms.py::AtivoForm.clean_patrimonio`): campo fica opcional no formulário; em branco, gera `PREFIXO-NNNNNN` sequencial por categoria e tenant; código digitado manualmente é validado por unicidade. Também registrado como exceção auditada (mesmo motivo do `core/unidades.py`).
- **Unidade obrigatória no ativo** (`ativos/models.py::Ativo.unidade`, `PROTECT`): migration `ativos/0005_*` faz o backfill (cria "Unidade Principal" só nos tenants que precisam, reaproveita a única existente quando não há ambiguidade) antes de aplicar o `NOT NULL`.
- **Escopo de unidade aplicado em todas as telas** (`core/unidades.py::filtrar_por_unidade`, usado por `ativos/views.py::ativos_no_escopo` e equivalentes): lista de ativos, ficha, QR Code, busca por patrimônio, wizard de empréstimo, devolução, mapa, manutenção, agenda, beneficiários, dashboard, relatórios e Centro de Etiquetas. Fora do escopo responde 404, não 403. Telas avisam explicitamente quando o usuário não tem unidade atribuída.
- **Centro de Etiquetas** (`ativos/etiquetas.py`, `ativos/views.py::etiquetas_*`, `templates/ativos/etiquetas/*.html`): fila de impressão, filtros (categoria/unidade/status/só sem etiqueta), seleção em lote, três layouts, histórico agrupado por lote, "Última impressão"/"Quantidade de impressões"/"Reimprimir" na ficha. PDF pela caixa de impressão do navegador — decisão documentada em `docs/business-rules/etiquetas.md`.

- **Transferência entre unidades** (`ativos/services.py::transferir`): não altera o estado operacional, exige justificativa, bloqueada para ativo emprestado, guarda origem e destino por nome. Tipo `recuperacao` foi separado de `transferencia`, que antes acumulava os dois significados.
- **Dashboard por unidade** (`ativos/selectors.py::resumo_por_unidade`) e indicadores agregados no banco em consulta única, no lugar de uma contagem por status e da materialização de todos os ativos em memória.
- **Extravio na UI e correção de manutenção em curso** (esta liberada ao Funcionário — quem está com o ativo na oficina é quem sabe corrigir).

Pendente (não implementado ainda)

- **"Localizar Ativo"**: tela dedicada única com escanear QR Code / pesquisar por patrimônio / categoria / nome. Hoje as três entradas existem, mas espalhadas (`scan`, lista de ativos, mapa) em vez de reunidas numa porta de entrada só.
- **Leitura de QR Code por câmera**: a rota de resolução já é o backend completo do fluxo; falta a biblioteca client-side de leitura (`Permissions-Policy` já libera `camera=(self)`).

QR Code (`Ativo.qr_token`, gerado por `gerar_qr_token()`) e o histórico completo por ativo (timeline, movimentações, fotos, manutenções, empréstimos, devoluções) já existiam antes desta spec e continuam válidos sem alteração.